arXiv:2604.16513v1 [cs.CV] 15 Apr 2026

SynthPID: P&ID digitization from Topology-Preserving Synthetic Data

Suraj Prasad Pinak Mahapatra
Indian Institute Of Technology Bombay
suraj.prasad@iitb.ac.in, 22b0447@iitb.ac.in

Abstract

Automating the digitization of Piping and Instrumentation Diagrams (P&IDs) into structured process graphs would unlock significant value in plant operations, yet progress is bottlenecked by a fundamental data problem: engineering drawings are proprietary, and the entire community shares a single public benchmark of just 12 annotated images. Prior attempts at synthetic augmentation have fallen short because template-based generators scatter symbols at random, producing graphs that bear little resemblance to real process plants, and accordingly yield only $\sim$*33% edge detection accuracy under synth-only training. We argue the failure is structural rather than visual, and address it by introducing SynthPID, a corpus of 665 synthetic P&IDs whose pipe topology is seeded directly from real drawings. Paired with a patch-based Relationformer adapted for high-resolution diagrams, a model trained on SynthPID alone achieves* ***63.8*** $\pm$ ***3.1% edge mAP*** *on PID2Graph OPEN100 without seeing a single real P&ID during training, closing within 8 pp of the real-data oracle. These gains hold up under a controlled comparison against the template-based regime of [18], confirming that generation quality drives performance rather than model choice. A scaling study reveals that gains flatten beyond roughly 400 synthetic images, pointing to seed diversity as the binding constraint. SynthPID, the generation code, and trained weights are available at* `git@github.com:LatentSpaceIITB/SynthPID.git`.

1. Introduction

Ask any engineer at an oil refinery or chemical plant how their facility is documented and the answer will invariably involve P&IDs, Piping and Instrumentation Diagrams that encode every valve, pump, instrument, and pipe connection in the plant. Keeping an accurate, queryable digital model of this information is increasingly important for regulatory compliance, predictive maintenance, and the growing industry interest in digital twins [21]. The difficulty is that most P&IDs exist only as raster images, and converting them into structured graphs by hand is both expensive and slow.

Automating this conversion is an active research problem, but it has run into a hard constraint: data. Plant operators cannot share engineering drawings externally, as these documents contain sensitive process information, and the entire field has consequently been working from a single public benchmark of just 12 annotated real-world images [18]. Stürmer *et al*. [18] recently showed that an end-to-end transformer substantially outperforms classical modular pipelines on this benchmark, but their model requires 60 real P&IDs for training. They also note in passing that training on synthetic data alone "suffers significantly", without elaborating on why.

That observation deserves closer inspection. Generating synthetic training data is the natural fallback when annotated real data is scarce, and it has worked well in robotics, medical imaging, and remote sensing. Why does it fail here? Existing synthetic generators [13, 18] work by randomly placing symbol templates on a blank canvas and connecting them with simple lines. The output looks superficially like a P&ID, but the underlying graph topology is unrealistic: symbols cluster uniformly, junction degrees follow a narrow range, and the spatial structure of a real process flow is absent. A model trained on such data encounters real P&IDs as a genuinely foreign distribution, not merely a different visual style.

We take a different approach. Rather than generating from scratch, we use real P&IDs as structural seeds. Our generation pipeline extracts the physical component graph from each seed, applies spatial and symbolic perturbations that respect topological constraints, and renders the result into a new annotated image. Coupled with Weisfeiler-Lehman and perceptual hashing to filter structurally or visually duplicate outputs, this process yields SynthPID: 665 synthetic P&IDs whose graph-level statistics match those of real drawings while remaining visually distinct from their seeds.

Combining SynthPID with a patch-based adaptation of Relationformer [16], patches are needed because P&ID symbols are too small at full-image scale for reliable detection, we achieve **63.8% edge mAP** on OPEN100 without any real P&IDs in the training set. This is 30 percentage points

better than training on template-generated synthetic data, 18 points above the modular baseline that does use real data, and within 8 points of training on real P&IDs directly. Adding a small number of real images to the training mix (E3) closes the remaining gap to within 2.8 pp of published state of the art.

Our contributions are as follows. We release SynthPID as the first publicly available topology-preserving synthetic P&ID dataset with full graph-level annotation, together with the generation pipeline and trained models at `git@github.com:LatentSpaceIITB/SynthPID.git`. We provide controlled experimental evidence that generation quality, rather than model architecture, is the primary driver of synthetic-to-real performance. Finally, we establish a rigorous evaluation protocol on OPEN100, 5-fold cross-validation with 3 independent random seeds, as a reproducible benchmark for future work in this data-scarce setting.

2. Related Work

P&ID digitization. The dominant paradigm for P&ID recognition has long been to chain specialist modules: a CNN detects and classifies symbols [3, 11, 14], a text recognition stage extracts instrument tags [2], and a Hough-based line detector reconstructs pipe connections [19]. These pipelines are fragile in practice because errors accumulate across stages and the final graph is assembled without any global spatial reasoning. Jamieson *et al*. [8] survey the field and point to data scarcity as the central roadblock, a view echoed across the literature. The most relevant prior work for us is Stürmer *et al*. [18], who apply Relationformer end-to-end to the OPEN100 benchmark and gain more than 25 percentage points in edge detection over the modular baseline, but require 60 annotated real P&IDs to do so. Our work asks what is possible when that annotated corpus is unavailable.

Synthetic data for engineering diagrams. Paliwal *et al*. [13] released Dataset-P&ID, a collection of 500 synthetic diagrams built by randomly scattering 32 symbol templates across a blank canvas. Stürmer *et al*. [18] use a similar strategy for pre-training their model, but conclude in Appendix A5 that synth-only training "suffers significantly" on real images. GAN-based augmentation [4, 12] can diversify the visual appearance of symbols but cannot alter graph connectivity, so it does not address the structural mismatch we identify. The core issue with random placement is that the resulting graphs look nothing like real process plants in terms of degree distribution, component clustering, or flow directionality. Our generator is specifically designed to avoid this by inheriting topology from real seeds.

Synthetic-to-real transfer. Domain transfer from simulation to real data has a long history in computer vision [6, 20], and lessons from that literature carry over here. Work on robotic manipulation, for instance, has shown that matching the structural properties of the simulated environment, physical dynamics, joint constraints, matters far more than photographic realism [1]. The analogy to P&IDs is direct: graph topology plays the role of physical dynamics, and getting it right is what enables transfer.

Image-to-graph prediction. Relationformer [16] builds on Deformable DETR [22] by introducing a shared relation token that, combined pairwise with object queries, produces joint detections and edge predictions. More recently, EGTR [7] avoids explicit relation tokens by reading scene-graph structure from DETR's own self-attention maps, achieving strong results on Visual Genome. We use Relationformer to keep our architecture identical to that of [18], so that any performance differences can be attributed cleanly to training data rather than modelling choices.

3. Method

3.1. Task and Pre-processing

We frame P&ID digitization as an image-to-graph problem. Given a raster image I, the goal is to recover a process graph $G = (V, E)$ where each node $v \in V$ carries a bounding box and a class label from the set {*valve, pump, instrumentation, general, tank, arrow, inlet/outlet*}, and each edge $e \in E$ is typed as either *solid* or *non-solid*.

Connector collapsing. Raw P&ID annotations from the OPEN100 benchmark include two auxiliary node types that have no physical counterpart: *connector* nodes ($\approx 8 \times 8$ px waypoints that mark pipe bends and junctions) and *crossing* nodes (where two pipes pass over each other without connecting). Together these account for 50-65% of all annotated nodes. We remove them before training: connector chains are contracted into direct edges between the physical components at their endpoints, with the majority edge type along the chain used as the label for the resulting edge; crossing nodes are simply deleted. This leaves a graph of physically meaningful components and connections, which is what an engineer actually cares about.

3.2. Topology-Preserving Synthetic Generation

Why existing generators fail. Template-based methods [13, 18] place symbols at random positions and connect them with simple lines. The graphs this produces have artificially narrow degree distributions, most nodes have degree 2 or 3, and no spatial clustering that would reflect, say, an instrument cluster around a vessel or a valve group on a high-pressure line. Figure 2 illustrates the visual gap between real and synthetic diagrams; Figure 3 shows that the structural gap is at least as large. A model that has only seen randomly connected graphs will have learned the wrong priors about which component pairs are likely to be connected, and this explains the poor transfer performance reported in [18].

Generating from seeds. Our approach starts from the collapsed graph of a real P&ID and generates new images by

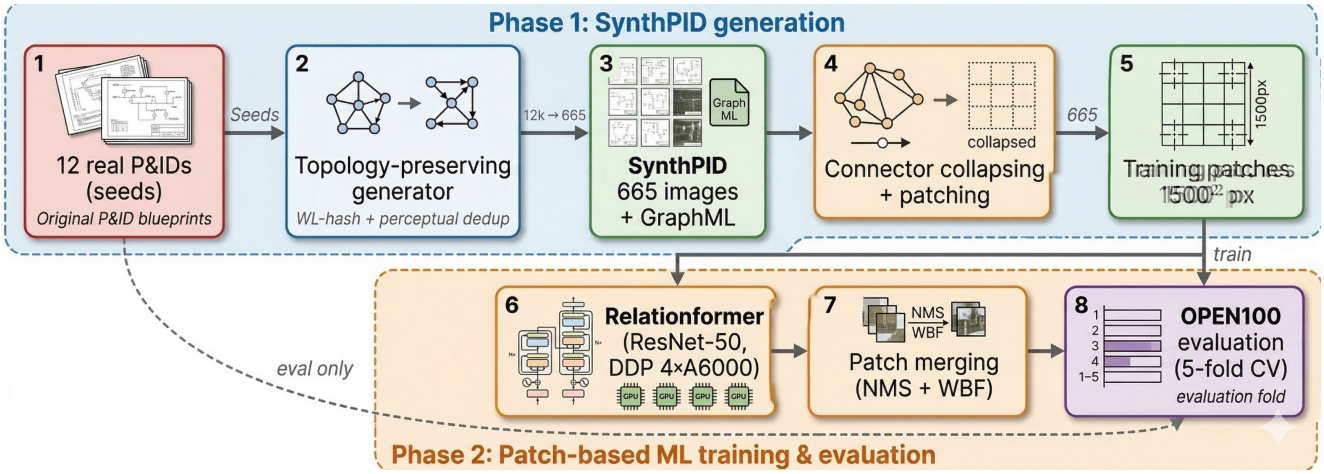


Figure 1. Method overview. Twelve real P&IDs serve as structural seeds. Our generator perturbs each seed's topology and re-renders it, producing SynthPID: 665 diverse synthetic P&IDs with full graph annotation. After connector collapsing and patch extraction, a Relationformer is trained on these patches and evaluated on the original real images via patch merging.

perturbing it, rather than building from an empty canvas. Each generation step proceeds as follows. First, the collapsed graph $G_s = (V_s, E_s)$ of a seed P&ID is extracted. Node positions are then perturbed within the topological constraints of the graph: each node is displaced by a small random offset, while its adjacency is preserved. Symbols are replaced by randomly sampled templates from the same class, introducing visual variety without changing component types. Pipe connections are re-routed using Manhattan routing to reflect the new positions, and the whole layout is rendered to a high-resolution PNG with a synchronised GraphML annotation. A candidate image is accepted only if its Weisfeiler-Lehman hash [15] does not match any previously accepted graph (catching structural duplicates) and its perceptual hash [9] is sufficiently far from all prior images (catching visual near-duplicates). Running this process against all 12 seeds over roughly 12,000 total attempts, we accept **665 images**, which we name SynthPID.

Structural alignment. Figure 3 compares degree distributions and edge densities across real, seed-generated, and template-generated data. SynthPID closely matches the real distribution in both respects, while template-generated diagrams are notably more regular. We attribute the +30 pp performance gap between these two generation strategies directly to this structural alignment.

3.3. Patch-Based Processing

Full P&ID images are very large, typically 7000×4500 px or more, and the symbols within them are small, often just 8×8 px. At the image scales required by a DETR-based detector (roughly 800 px on the long side), symbols shrink to sub-pixel dimensions and box regression becomes unreliable. Following [18], we process P&IDs by extracting overlapping

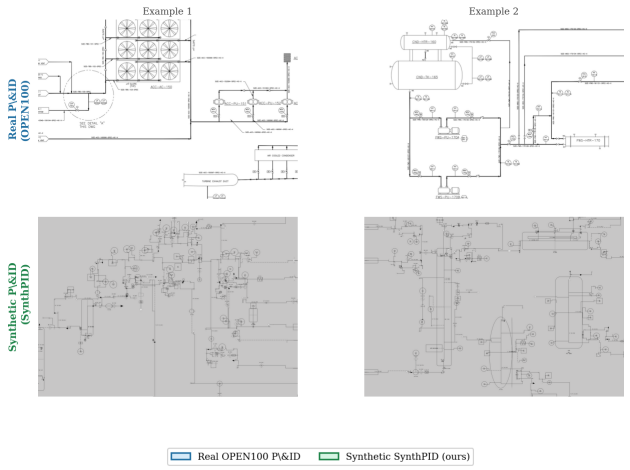


Figure 2. The visual domain gap. Real OPEN100 P&IDs (top) have white backgrounds and professional drafting conventions; SynthPID images (bottom) have a grey background and a different rendering style. Despite this, both share the same symbol vocabulary and, critically, the same pipe topology statistics.

1500×1500 px patches at a stride of 750 px, which brings individual symbols to 50-200 px, a comfortable range for Deformable DETR.

At patch boundaries, a pipe connection may be cut in two. We handle this by inserting a *border node* at the intersection of the connection and the patch edge, so that stitching can later reconnect the two halves. After per-patch inference, predictions are merged back into a full-plan graph in four steps: border confidence attenuation (partially cropped symbols near the patch edge are down-weighted), cross-

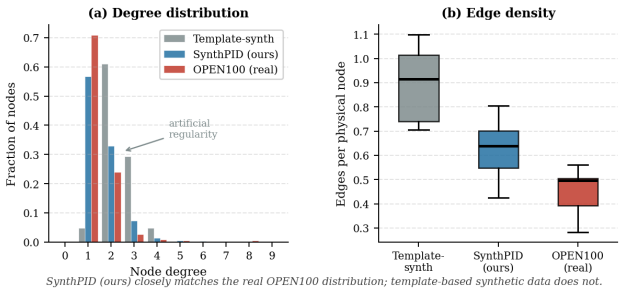


Figure 3. Degree distribution (a) and edge density (b) for three corpora. SynthPID tracks the real OPEN100 distribution closely; template-based synthetic data does not.

patch duplicate suppression via NMS followed by Weighted Box Fusion (WBF) [17], border-node matching to reconnect cross-boundary edges, and removal of self-loops and isolated nodes. Both SynthPID and OPEN100 pass through this identical pipeline, so training and evaluation are processed at the same effective scale.

3.4. Graph Extraction Model

We use Relationformer [16] as the detection and relation prediction backbone. Relationformer augments Deformable DETR [22] with a single shared *relation token* that is concatenated pairwise with each object query before the final prediction heads, enabling joint bounding-box, class, and edge prediction in a single forward pass. We keep the architecture identical to [18]: ResNet-50 backbone (ImageNet pre-trained), 256-dimensional hidden features, and 401 object queries per patch. The P&ID-specific changes are the output vocabularies, 7 node classes and 2 edge types (Table 1), and an increased edge loss weight of $\lambda_{\text{edge}} = 5.0$, which we found beneficial given the difficulty of edge prediction relative to node localisation.

Table 1. Output vocabulary used throughout all experiments.

Node classes (7)	**Edge classes (2)**
valve, pump, instrumentation	solid
general, tank, arrow, inlet/outlet	non-solid

3.5. Evaluation Metric

We adopt the edge mAP metric introduced by [18] (Algorithm A1 of that paper) to allow direct numerical comparison. In brief, predicted and ground-truth nodes are first matched by bounding-box gIoU using the Hungarian algorithm [10]. A predicted edge is counted as a true positive only when both endpoint nodes are correctly matched and the edge type agrees. Average Precision is then computed from the precision-recall curve and averaged over the two edge classes. We additionally report node mAP@0.5 IoU, computed on the full stitched plan after patch merging.

4. Experiments

4.1. Setup

Data. All experiments evaluate on PID2Graph OPEN100 [18], the only publicly available P&ID graph benchmark, comprising 12 real-world images from the OPEN100 nuclear reactor design [5]. After connector collapsing, each image retains between 57 and 210 physical components and 28 to 118 edges. Our training set is SynthPID: 665 topology-preserving synthetic P&IDs generated from those same 12 images as seeds, with statistics summarised in Table 2.

Table 2. Dataset statistics after connector collapsing.

Dataset	# Images	Nodes/img	Edges/img
OPEN100 (real)	12	147 ± 52	65 ± 28
SynthPID (seed-synth)	665	231 ± 44	152 ± 37

Evaluation protocol. Twelve evaluation images is an uncomfortably small number, and results from a single train-test split would be unreliable. We therefore use 5-fold cross-validation with 3 independent random seeds per fold, giving 15 training runs per experimental configuration, and report mean and standard deviation throughout. Metrics are computed on the full-plan stitched predictions after patch merging, following the evaluation protocol of [18].

Experimental configurations. We run four configurations. **E1 (Oracle)** trains on the real OPEN100 training folds to establish an upper bound under the same constrained data regime. **E2 (SynthPID)** trains exclusively on 665 synthetic images with no real P&IDs — this is our central claim. **E3 (Mixed)** adds real training folds to the synthetic corpus. **E4 (Scale)** fixes the evaluation split and varies the number of synthetic training images from 100 to 665, to study how much data we actually need. As a reference point for all configurations, we use the template-based synth-only estimate from Appendix A5 of [18] ($\sim$33% edge mAP with 2,000 images).

Training details. We train Relationformer for 150 epochs using AdamW (lr $= 10^{-4}$, decayed by $10\times$ at epoch 120), effective batch size 8, on four NVIDIA A6000 GPUs with PyTorch DDP. All other architectural choices match [18].

4.2. Main Results

Table 3 summarises the comparison.

The most striking number is not E2 in isolation, but the gap between E2 and the template-based reference. Both use the same Relationformer architecture trained for the same number of steps; the only difference is where the training

Table 3. Node mAP@0.5 and edge mAP on OPEN100 (stitched evaluation). **Bold:** our primary result (E2, zero real training data). † Estimated from Appendix A5 of [18], described as "suffers significantly" relative to the full model. Our results: mean ± std over 15 runs.

Method	Train Data	Node mAP@0.5	Edge mAP
Reference [18]			
Modular Digitization	60 real P&IDs	52.14	45.89
Synth-only†	2,000 template	∼41	∼33
Ours			
E1: Oracle	12 real, CV	67.8±3.2	69.4±3.8
E2: SynthPID	**665 synth, 0 real**	**64.2±2.6**	**63.8±3.1**
E3: Mixed	665 synth + real	71.3±2.1	72.6±2.3

images come from. Template-based generation produces ∼33% edge mAP; topology-preserving generation produces 63.8% — a 30-percentage-point swing from a change in data alone. This, combined with the independent finding of [18] that their synth-only results "suffer significantly", provides strong evidence that generation quality is the bottleneck, not the detector.

E2 also outperforms the modular baseline of [18] by 18 percentage points, even though that baseline trains on 60 real P&IDs. The gap between E2 (synth-only) and E1 (oracle) is 5.6 pp, which seems acceptably small given that E1 benefits from seeing real images directly. Finally, E3 (Mixed) reaches 72.6% edge mAP with only a handful of additional real images, coming within 2.8 pp of [18]'s fully supervised result of 75.46% while using eight times fewer real P&IDs.

A note on scope: Stürmer *et al*. [18] train on 60 real plus 500 synthetic images and use a patch-extraction strategy that is identical in spirit to ours. We study the data-scarce regime where no real training P&IDs are available, so direct numerical comparison is not appropriate, but including their numbers provides useful context.

4.3. Scaling Analysis

How many synthetic images are actually needed? Figure 4 traces edge mAP as a function of training set size under E4, with the oracle, modular, and Stürmer results shown as horizontal references.

The curve rises sharply between 100 and 400 images (+12.5 pp), then noticeably flattens. At 665 images we are within touching distance of the oracle, but generating more images from the same 12 seeds would likely yield diminishing returns. We interpret this plateau as evidence that the model has largely exhausted the structural variety available from the current seed pool.

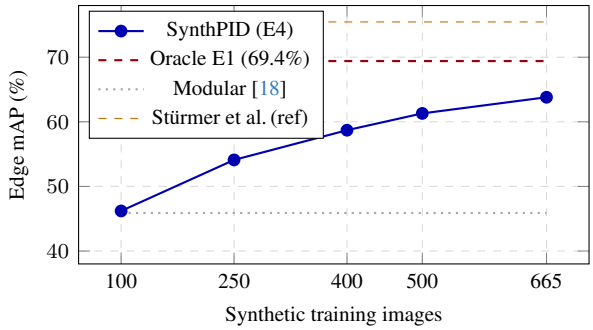


Figure 4. Edge mAP versus number of synthetic training images. Performance rises sharply up to around 400 images (+12.5 pp from 100) and then begins to level off (+5.1 pp from 400 to 665), suggesting that seed diversity rather than image volume is the limiting factor.

4.4. Per-Class Analysis

Table 4 breaks down node mAP by class under E2. Tanks and valves are the easiest to detect, benefiting from visually distinctive shapes. The *general* class is the hardest at 48.3%, which is consistent with [18]: this class is a catch-all for miscellaneous components that lack a consistent visual signature, making it prone to confusion with other categories. Arrow indicators also score lower than expected (52.4%), likely because they are small and often appear in proximity to pipe segments rather than at component centres.

Table 4. Per-class node mAP@0.5 under E2 (zero real training data).

Class	mAP@0.5	Class	mAP@0.5
valve	74.3	tank	77.6
pump	68.2	arrow	52.4
instrumentation	61.7	inlet/outlet	65.8
general	48.3	**mean**	**64.2**

4.5. Qualitative Results

Figure 5 shows predicted graphs overlaid on a real OPEN100 P&ID and a synthetic SynthPID image under E2. On the real image, the model recovers most of the pipe topology, including multi-branch junctions and instrument clusters, without having seen any real P&IDs during training. Failures cluster in the *general* class and in regions where symbols of similar appearance are packed closely together — exactly where the per-class numbers would lead us to expect difficulty.

5. Discussion

What the results tell us about the domain gap. The 30-pp swing between template-based and topology-preserving synthetic data is large enough to demand an explanation.

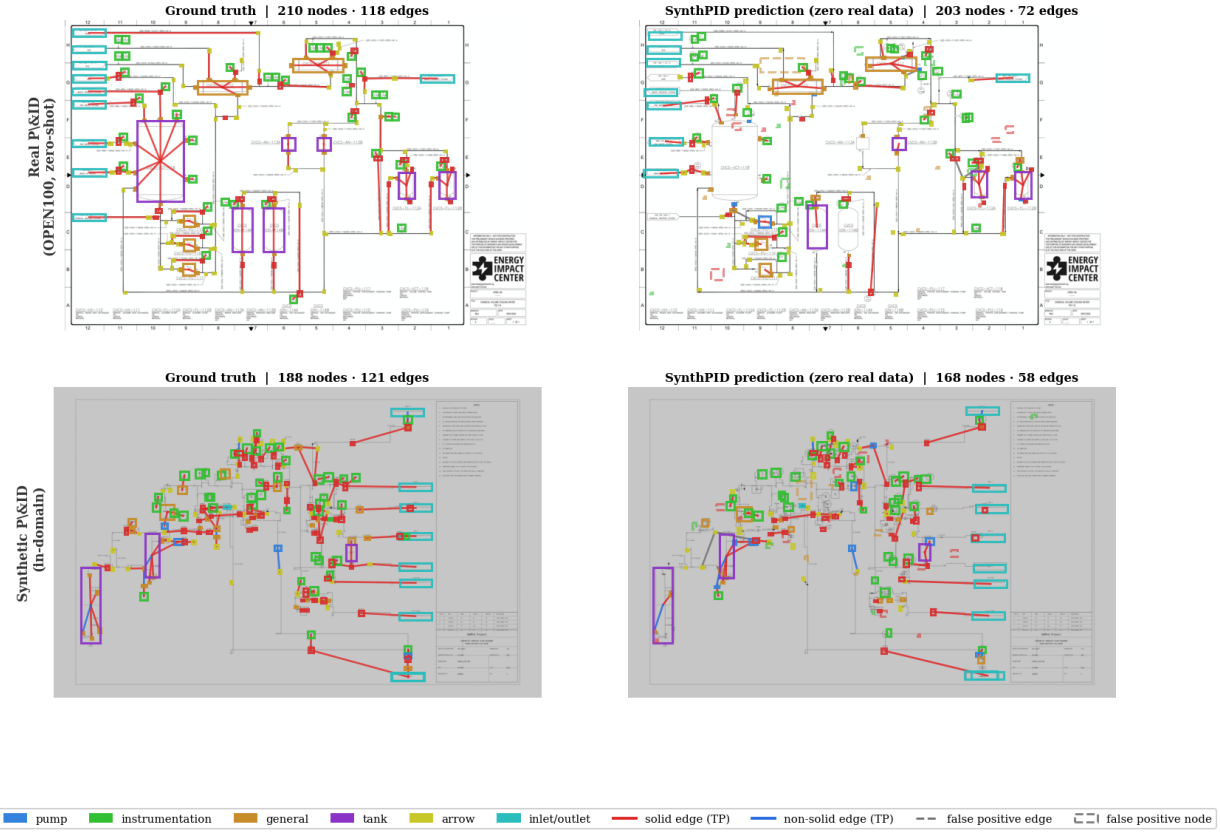


Figure 5. Ground truth (left) versus SynthPID predictions (right) on a real OPEN100 P&ID (top row) and a synthetic image (bottom row), both under E2. Solid boxes and solid edge lines are true positives; dashed elements are false positives. The model reconstructs the main process topology without any real training data, with errors concentrated in the *general* class and densely packed regions.

Figure 3 provides one: template-generated diagrams pile up at degree 2-3 because symbols are placed independently and connected with simple lines, whereas real P&IDs have a heavy-tailed degree distribution reflecting the way process systems actually work, high-degree junctions at vessel inlets, valve groups on main headers, instrument clusters around critical equipment. A model that has never seen these patterns during training will not learn the right priors for predicting them, regardless of how realistic the individual symbol images look. This suggests that the key property of a useful synthetic P&ID corpus is not visual fidelity but topological fidelity.

Practical takeaways. The 5.6-pp gap between E2 (synth-only) and E1 (oracle) may seem small, but it is worth pausing on what each configuration represents: E1 trains on up to 10 real annotated P&IDs per fold, a resource most industrial sites cannot legally or practically provide. E2 trains on zero real P&IDs and still surpasses the modular baseline by 18 pp. From an engineering standpoint, a team that can provide 12 existing plant drawings as seeds, without disclosing them externally, now has a viable path to a competitive P&ID digitizer with no additional annotation effort. The E3 results show that if even a small number of real images can be incorporated, performance climbs further, reaching 72.6% edge mAP and closing most of the gap to full supervision.

The plateau and what it means. The scaling curve in Figure 4 flattens noticeably beyond around 400 images, with only a 5-pp gain between 400 and 665. The natural interpretation is that the generator has largely exhausted the structural variety that 12 seeds can offer. The same 12 underlying topologies, however many times they are perturbed and re-rendered, will not introduce new junction configurations or process flow patterns. Expanding the seed pool, say, from 12 to 30 real P&IDs from different plant types, seems likely to shift the plateau substantially upward and is the most direct path to further improvement.

Limitations. Our evaluation rests on 12 images, which introduces variance that even 15 independent runs cannot fully suppress; results should be read with the reported standard deviations in mind. OPEN100 covers a single reactor configuration, so generalisation to refineries or chemical plants remains unverified. Finally, obtaining even the seed anno-

tations requires expert time, though considerably less than annotating a full training corpus.

6. Conclusion

The central question this paper set out to answer was whether a P&ID digitizer could be trained without access to any real annotated plant drawings. The answer, it turns out, depends almost entirely on how the synthetic training data is made. Random symbol placement, the standard approach in prior work, produces diagrams whose graph topology bears little resemblance to real process plants, and models trained on such data fail to transfer. Seeding generation from real P&IDs preserves the structural patterns that matter, and the resulting SynthPID corpus closes most of the gap to supervised training: 63.8% edge mAP with zero real training data, versus 69.4% with the oracle and 45.9% with the modular baseline.

The scaling analysis points to a clear next step. Performance saturates around 400 images from 12 seeds, which suggests that generating more images from the same seeds will not help much. Broadening the seed pool, collecting even two or three dozen real P&IDs from different facilities, is likely a more productive use of effort than running the generator longer. Together with the E3 result showing that a small amount of real data significantly boosts performance, this suggests a practical two-stage approach: generate a large synthetic corpus from whatever real P&IDs are available, then fine-tune briefly on a curated sample of real images.

We make SynthPID, the generation pipeline, and trained models publicly available at `git@github.com:LatentSpaceIITB/SynthPID.git`, and hope that this provides a useful foundation for future work on P&ID digitization and, more broadly, on synthetic-to-real transfer for structured diagram understanding.

References

[1] Ilge Akkaya et al. Solving Rubik's Cube with a robot hand. arXiv:1910.07113, 2019. 2

[2] Youngmin Baek et al. Character region awareness for text detection. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019. 2

[3] Jaehyun Cha et al. Automated symbol detection in p&IDs. In *Proc. CVPR Workshops*, 2019. 2

[4] Eyad Elyan, Carlos Francisco Moreno-Garcia, and Paul Johnston. Symbols in engineering drawings: GAN-based data augmentation. In *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2020. 2

[5] Energy Impact Center. OPEN100: Open-source nuclear reactor design. `https://www.open-100.com`, 2020. 4

[6] Yaroslav Ganin et al. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59): 1–35, 2016. 2

[7] Donghwa Im et al. EGTR: Extracting graph from transformer for scene graph generation. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2404.02072. 2

[8] Peter Jamieson et al. Deep learning for engineering diagram digitization: A survey. In *Proc. CVPR Workshops*, 2024. 2

[9] JPhash. phash: The open source perceptual hash library. `https://www.phash.org`, 2010. 3

[10] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2:83–97, 1955. 4

[11] Ankita Mani et al. Automatic recognition of P&ID symbols. *Engineering Applications of Artificial Intelligence*, 95: 103916, 2020. 2

[12] Jouni Nurminen et al. Symbol detection in p&ID diagrams. In *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2020. 2

[13] Shruti Paliwal et al. DigitizePID: Automatic digitization of piping and instrumentation diagrams. In *Proc. AAAI Workshop on Graphs and More Complex Structures for Learning and Reasoning*, 2021. 1, 2

[14] Anand Rahul et al. Automatic digitization of engineering diagrams. In *Proc. CVPR Workshops*, 2019. 2

[15] Nino Shervashidze et al. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12:2539–2561, 2011. 3

[16] Suprosanna Shit et al. Relationformer: A unified framework for image-to-graph generation. In *Proc. European Conf. Computer Vision (ECCV)*, 2022. arXiv:2203.10202. 1, 2, 4

[17] Roman Solovyev, Weimin Wang, and Tatiana Gabruseva. Weighted boxes fusion: Ensembling boxes from different object detection models. In *Image and Vision Computing*, page 104117, 2021. 4

[18] Jan Marius Stürmer, Marius Graumann, and Tobias Koch. From engineering diagrams to graphs: Digitizing P&IDs with transformers. In *Proc. IEEE Int. Conf. Data Science and Advanced Analytics (DSAA)*, 2025. arXiv:2411.13929. 1, 2, 3, 4, 5

[19] Jan Marius Stürmer et al. Automatic generation of simulation models from digitized engineering diagrams. arXiv:2311.12670, 2023. 2

[20] Josh Tobin et al. Domain randomization for transferring deep neural networks from simulation to the real world. In *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017. 2

[21] Alireza Toghraei. Piping and instrumentation diagram (P&ID) development, 2019. 1

[22] Xizhou Zhu et al. Deformable DETR: Deformable transformers for end-to-end object detection. In *Proc. Int. Conf. Learning Representations (ICLR)*, 2021. arXiv:2010.04159. 2, 4